Object类、最终类、内部类和匿名类

主讲老师：申雪萍

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

主要内容

- Object类
- 操作符“==”与对象的equals()方法
- 最终方法、最终类
- 内部类
- 匿名类

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

Java类的祖先类:Object类

北京航空航天大學
COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

Object类

- Object是Java中所有类的父类、超类、基类，位于继承树的最顶层，相当于是Java中的“万类之王”。
- 任何一个没有显式地继承别的父类的类，都会直接继承Object，否则就是间接地继承Object，并且任何一个类也都会享有Object提供的方法。同时Java中的所有类对象，包括数组，也都要实现这个类中的方法。
- 基于多态的特性，该类可以用来代表任何一个类，因此允许把任何类型的对象赋给 Object类型的变量，也可以作为方法的参数、方法的返回值。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

Object类提供的几个关键的方法

方法摘要	
protected Object	clone() 创建并返回此对象的一个副本。
boolean	equals(Object obj) 指示其他某个对象是否与此对象“相等”。
protected void	finalize() 当垃圾回收器确定不存在对该对象的更多引用时，由对象的垃圾回收器调用此方法。
Class<?>	getClass() 返回此 Object 的运行时类。
int	hashCode() 返回该对象的哈希码值。
void	notify() 唤醒在此对象监视器上等待的单个线程。
void	notifyAll() 唤醒在此对象监视器上等待的所有线程。
String	toString() 返回该对象的字符串表示。
void	wait() 在其他线程调用此对象的 notify() 方法或 notifyAll() 方法前，导致当前线程等待。
void	wait(long timeout) 在其他线程调用此对象的 notify() 方法或 notifyAll() 方法，或者超过指定的时间量前，导致当前线程等待。
void	wait(long timeout, int nanos) 在其他线程调用此对象的 notify() 方法或 notifyAll() 方法，或者其他某个线程中断当前线程，或者已超过某个实际时间量前，导致当前线程等待。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

Object类的常用方法（1）

- **1.** `public final Class<?> getClass () { }`
  - 该方法用于获取对象运行时的字节码类型，得到该对象的运行时的真实类型。
  - 通常用于判断两个引用中**实际存储对象类型**是否一致。
  - 最主要应用：该方法属于Java的反射机制，其返回值是Class类型，例如 `Class c = obj.getClass();`。通过对象c，我们可以进一步获取该对象的所有成员方法，每个成员方法都是一个Method对象。我们也可以获取该对象的所有成员变量，每个成员变量都是一个Field对象。同样的，我们也可以获取该对象的构造函数，构造函数则是一个Constructor对象。

Object类的常用方法（1）

- **2.** public int hashCode() { }
  - 返回该对象的**哈希码值**。*哈希值根据对象的地址或字符串或数字使用hash算法计算出来的**int类型的数值**。*
  - 在Object类中，hashCode的默认实现通常会返回对象的内存地址的某种形式（不能完全将哈希值等价于地址），具体的实现依赖JVM。
  - 提高具有哈希结构的容器的效率。
  - 如果两个对象相等（即equals返回true），那么它们的hashCode值也必须相等。

Object类的常用方法（1）

- 3. public String toString() { }
- **4.** public boolean equals(Object obj) { }
  - 默认实现为（this == obj），比较两个对象**地址**是否相同。也就是对象的hashCode值也必须相等。
  - 可进行**覆盖**，比较两个对象的**内容**是否相同。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

Object类的常用方法（2）

- 5.finalize()方法
- 6.线程中常用的方法
  - public final void wait() >>> 多线程中等待功能
  - public final native void notify() >>> 多线程中唤醒功能
  - public final native void notifyAll() >>> 多线程中唤醒所有等待线程的功能

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

Java类的祖先类:Object类

- 所有的Java类都直接或间接地继承了java.lang.Object类，Object类是所有Java类的祖先，在这个类中定义了所有的Java对象都具有的方法。
  - equals（Object obj）：比较两个对象是否相等。仅当被比较的两个引用变量指向同一对象时，equals()方法返回true。
  - toString()：返回当前对象的字符串表示。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

equals()和== 的比较

- The equals method for class Object implements the most discriminating possble equivalence relation on objects; **that is, for any reference values x and y,this method returns true if and only if x and y refer to the same object(x==y) has the value true).**
  - **判断是否为同一引用**
- ObjectTest.java

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

```java
package com.buaa.edu.cn;
class A {
    public String toString() {
        return "A is for ObjectTest class";
    }
}
class B {
}
public class ObjectTest {
    public static void main(String[] s) {
        A a1 = new A();
        A a2 = new A();
        A a3 = a1;
        B b1 = new B();
        System.out.println("a1.equals(a1) is " + a1.equals(a1));
        System.out.println("a1.equals(a2) is " + a1.equals(a2));
        System.out.println("a1.equals(a3) is " + a1.equals(a3));
        System.out.println(a1.toString());
        System.out.println("a1 is a instance of class "
                + a1.getClass().getName());
        // /默认的toString()输出包名加类名和堆上的首地址
        System.out.println(b1.toString());
        System.out.println("b1 is a instance of class "
                + b1.getClass().getName());
    }
}
```

```
a1.equals(a1) is true
a1.equals(a2) is false
a1.equals(a3) is true
A is for ObjectTest class
a1 is a instance of class com.buaa.edu.cn.A
com.buaa.edu.cn.B@15db9742
b1 is a instance of class com.buaa.edu.cn.B
```

默认的toString()输出包名加类名和堆上的首地址

a1.getClass().getName()输出包名加类名

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

操作符“==”与对象的equals()方法

- equals()方法是在Object类中定义的方法，它的声明格式如下：

  public boolean equals(Object obj)

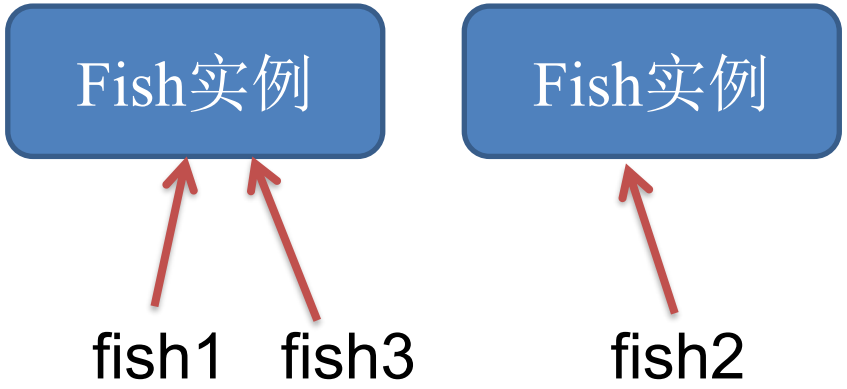
- Object类的equals()方法的比较规则为：当参数obj引用的对象与当前对象为同一个对象，就返回true，否则返回false：

```
public boolean equals(Object obj){
  if(this==obj)
        return true;
  else
        return false;
}
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

操作符“==”与对象的equals()方法

- 当操作符“==”两边都是引用类型变量时，这两个引用变量必须都引用同一个对象，结果才为true。




```
Food fish1=new Fish();
Food fish2=new Fish();
Food fish3=fish1;
System.out.println(fish1 ==fish2);  //打印false
System.out.println(fish1==fish3);  //打印true
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

操作符“==”与对象的equals()方法

- 在JDK中有一些类覆盖了Object类的equals()方法，它们的比较规则为：**如果两个对象的类型一致，并且内容一致，则返回true。判断的是长的像不像**
- 这些类包括：
  - **java.io.File、**
  - **java.util.Date、**
  - **java.lang.String、**
  - **包装类（如java.lang.Integer和java.lang.Double类等）。**

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

操作符“==”与对象的equals()方法

```
String str1=new String("Hello");
String str2=new String("Hello");

System.out.println(str1==str2);  //打印false
System.out.println(str1.equals(str2));  //打印true
```

小结：

- instanceof：判断是否为某类的一个实例
- Object类中：
  - 判断是否为同一引用：操作符“==”与对象的equals()方法，他们是等价的。

- ***注意：某些类例外，操作符“==”与对象的equals()方法不等价***

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

什么是hashCode

- hashCode就是对象的散列码，是根据对象的某些信息推导出的一个整数值，hashCode的默认实现通常会返回对象的内存地址的某种形式（不能完全将哈希值等价于地址），具体的实现依赖JVM。
- 通过散列码，哈希码用于确定对象在哈希表中的索引位置，可以提高检索的效率，主要用于在散列存储结构中快速确定对象的存储地址，如Hashtable、hashMap中

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

散列码可以提高检索的效率

- 如果集合中有10000个元素，如何快速高效的获取到相应的对象呢?
- 可以采用哈希算法（散列算法）来提高从集合中查找元素的效率。
- 每个对象可以计算出一个哈希码，可以将哈希码分组（使用不同的hash函数来计算），每组分别对应某个存储区域。
- 将数据按特定算法直接分配到不同区域上。将集合分成若干个存储区域，根据一个对象的哈希码就可以确定该对象应该存储在哪个区域，大大减少查询匹配元素的数量。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

equals()与hashCode()的联系

- 在Obeject类中，hashCode主要是用于查找使用的，而equals()是用于比较两个对象是否相等的。
- 如果两个对象的equals()结果为true，那么这两个对象的hashCode一定相同；
- 两个对象的hashCode()结果相同，并不能代表两个对象的equals()一定为true，Hash散列值有冲突的情况，虽然概率很低，只能够说明这两个对象在一个散列存储结构中；

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

为什么要重写 hashCode() 和 equals()？

- equals() 方法用于比较两个对象的内容是否相等。在Java中，默认实现是比较对象的引用，即比较两个对象是否指向内存中的相同位置。但通常，我们希望比较对象的内容是否相等。
- 鉴于这种情况，Object类中 equals() 方法的默认实现是没有实用价值的，所以通常都要重写。
- 而由于hashCode()与equals()具有联动关系（如果两个对象相等，则它们必须有相同的哈希码），所以equals()方法重写时，通常也要将hashCode()进行重写，使得这两个方法始终保持一致性。

```java
package com.pack1;

public class Test {
        public static void main(String[] args) {
                Person person1 = new Person();
                Person person2 = new Person();
                Person person3 = person2;

                System.out.println("person1 的 hashcode: " + person1.hashCode());
                System.out.println("person2 的 hashcode: " + person2.hashCode());
                System.out.println("person3 的 hashcode: " + person3.hashCode());
        }
}

class Person {

}
```

```
person1 的 hashcode: 1163157884
person2 的 hashcode: 1956725890
person3 的 hashcode: 1956725890
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

如果不重写 hashCode() 和 equals() 方法

```
class Person {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
 public class Main {
    public static void main(String[] args) {
        Person p1 = new Person("John", 25);
        Person p2 = new Person("John", 25);
        // 输出：false，默认比较的是内存地址
        System.out.println(p1.equals(p2));
        // 输出：false，hashCode 基于内存地址
        System.out.println(p1.hashCode() == p2.hashCode());
    }
}
```

默认情况下，equals() 和 hashCode() 是基于对象的内存地址比较的

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

重写 equals() 但不重写 hashCode()

```
class Person {
    String name; int age;
    Person(String name, int age) {
        this.name = name;
        this.age = age;}
    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Person person = (Person) obj;
        return age == person.age && name.equals(person.name);
    }}
public class Main {
    public static void main(String[] args) {
        Person p1 = new Person("John", 25);
        Person p2 = new Person("John", 25);
        System.out.println(p1.equals(p2)); // 输出：true，内容相同
        System.out.println(p1.hashCode() == p2.hashCode()); // 输出：false，hashCode 不相等
        // 放入 HashSet 中
        HashSet<Person> set = new HashSet<>();
        set.add(p1);
        set.add(p2);
        System.out.println(set.size()); // 输出：2，应该是1（因为 equals 相等），但 hashCode 不同导致重复
    }
}
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

重写 equals() 和 hashCode()

```
class Person {
    String name;int age;
    Person(String name, int age) {
        this.name = name;
        this.age = age; }
    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Person person = (Person) obj;
        return age == person.age && name.equals(person.name);
    }
    @Override
    public int hashCode() {
        return name.hashCode() + age; // 基于对象的属性计算 hashCode
    }}
public class Main {
    public static void main(String[] args) {
        Person p1 = new Person("John", 25);
        Person p2 = new Person("John", 25);
        System.out.println(p1.equals(p2)); // 输出：true
        System.out.println(p1.hashCode() == p2.hashCode()); // 输出：true
        // 放入 HashSet 中
        HashSet<Person> set = new HashSet<>();
        set.add(p1);
        set.add(p2);
        System.out.println(set.size()); // 输出：1，说明 p1 和 p2 被认为是相同的对象
    }}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

重写equals一定要重写hashCode吗？（建议）

- 如果仅仅是为了比较两个对象是否相等只重写equals就可以；
- 如果你使用了hashSet、hashMap等容器，为了避免加入重复元素，或者查找元素，就一定要同时重写两个方法。
- 如果自定义对象作为 Map 的键，那么必须重写 hashCode 和 equals 。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

小结

- equals() 用于比较两个对象是否相等，通常基于对象的内容来比较。
- hashCode() 用于生成对象的哈希值，主要用于提高基于哈希表数据结构的效率。
- 如果重写了 equals()，务必也要重写 hashCode()，以确保相等的对象拥有相同的哈希值，从而保证哈希表中的正常操作。
- 如果两个对象 equals() 返回 true，它们的 hashCode() 必须相同；但 hashCode() 相同的对象不一定 equals() 返回 true。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

Objects 与 Object 区别

- Object 是 Java 中所有类的基类，位于java.lang包。所有对象（包括数组）都实现此类的方法。
- Objects 【public final class Objects extends Object】是 Object 的工具类，位于java.util包。它从jdk1.7开始才出现，被final修饰不能被继承，拥有私有的构造函数。此类包含static实用程序方法，用于操作对象或在操作前检查某些条件。
  - null 或 null方法 ；
  - 用于计算对象的哈希代码；
  - 返回对象的字符串；
  - 比较两个对象，以及检查索引或子范围值是否超出范围。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

Java的常用类库——java.util.Objects

1、 equals
2、 deepEquals
    Objects中的equals和deepEquals二者方法的区别
3、 isNull
4、 nonNull
5、 requireNonNull
    requireNonNull(T obj)
    requireNonNull(T obj, String message)
6、 requireNonNull系列
    requireNonNull(T obj, Supplier messageSupplier)
    requireNonNullElse
    requireNonNullElseGet
    小结
7、 hashCode
8、 hash
9、 toString
    toString(Object o)
    toString(Object o, String nullDefault)
10、 compare
11、 Index系列
    checkIndex
    checkFromToIndex
    checkFromIndexSize

Java的常用类库——java.util.Objects

变量和类型	方法	描述
static int	checkFromIndexSize(int fromIndex, int size, int length)	检查是否在子范围从 fromIndex（包括）到 fromIndex + size（不包括）是范围界限内 0（包括）到 length（不包括）。
static int	checkFromToIndex(int fromIndex, int toIndex, int length)	检查是否在子范围从 fromIndex（包括）到 toIndex（不包括）是范围界限内 0（包括）到 length（不包括）。
static int	checkIndex(int index, int length)	检查 index是否在 0（含）到 length（不包括）范围内。
static <T> int	compare(T a, T b, Comparator<? super T> c)	如果参数相同则返回o，否则返回 c.compare(a, b)。
static boolean	deepEquals(Object a, Object b)	返回 true如果参数是深层相等，彼此 false其他。
static boolean	equals(Object a, Object b)	返回 true如果参数相等，彼此 false其他。
static int	hash(Object... values)	为一系列输入值生成哈希码。
static int	hashCode(Object o)	返回非的哈希码 null参数，o为 null的论点。
static boolean	isNull(Object obj)	返回 true如果提供的参考是 null，否则返回 false。
static boolean	nonNull(Object obj)	返回 true如果提供的参考是非 null否则返回 false。
static <T> T	requireNonNull(T obj)	检查指定的对象引用是否不是 null。
static <T> T	requireNonNull(T obj, String message)	检查指定的对象引用是否为null，如果是，则抛出自定义的NullPointerException。
static <T> T	requireNonNull(T obj, Supplier<String> messageSupplier)	检查指定的对象引用是否为null，如果是，则抛出自定义的NullPointerException。
static <T> T	requireNonNullElse(T obj, T defaultObj)	如果它是非 null，则返回第一个参数，否则返回非 null第二个参数。
static <T> T	requireNonNullElseGet(T obj, Supplier<? extends T> supplier)	如果它是非 null，则返回第一个参数，否则返回非 null值 supplier.get()。
static String	toString(Object o)	返回调用的结果 toString对于非 null参数， "null"为 null的说法。
static String	toString(Object o, String nullDefault)	如果第一个参数不是 null，则返回在第一个参数上调用 toString的结果，否则返回第二个参数。

https://blog.csdn.net/weixin_4631

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

主要内容

- Object类
- 操作符“==”与对象的equals()方法
- **最终方法、最终类**
- 内部类
- 匿名类

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

最终方法，最终类的必要性

- 因为继承破坏了封装性，换句话说，子类依赖于父类的实现细节,**继承很容易改变父类实现的细节，**即使父类整体没有问题，也有可能因为子类细节实现不当，而破坏父类的约束。
- **所以父类中能写成final尽量写成final**

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

最终类

- 如果一个类没有必要再派生子类，通常可以用final关键字修饰，表明它是一个最终类。

```
1. Public final class Math{}
2. Public final class String{}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

最终方法

- 最终方法：用关键字final修饰的方法称为最终方法。

- 最终方法既不能被覆盖，也不能被重载，它是一个最终方法，其方法的定义永远不能改变

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

final方法和final类

- final类中的方法可以不声明为final方法，但实际上final类中的方法都是隐式的final方法
- final修饰的方法不一定要存在于final类中。
- **定义类头时，abstract和final不能同时使用**
- 访问权限为private的方法默认为final的

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

常量

一. 常量：在程序运行过程中，其值不变的量。

二. Java中的常量使用关键字final修饰。

三. final既可以修饰简单数据类型，也可以修饰复合数据类型。

四. final常量可以在声明的同时赋初值，也可以在构造函数中

五. 复合数据类型常量可以是Java类库定义的复合数据类型，也可以是用户自定义的复合数据类型

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

常量声明格式

格式：final 数据类型 常量名=值

例如：final double PI=3.1415926;

final String str=“Hello World”;

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

注意事项

一. 简单数据类型常量其值一旦确定，就不能被改变。

二. **复合数据类型常量指的是引用不能被改变，而其具体的值是可以改变的。**

三. 常量既可以是局部常量，也可以是类常量和实例常量。如果是类常量，在数据类型前加static修饰（由所有对象共享）。如果是实例常量，就不加static修饰。

四. 常量名一般大写，多个单词之间用下划线连接。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

示例代码

```
class PersonA {
    String name;
    String sex;
    int age;
    final static double PAI = 3.1415926; // 静态常量
    final double ID; // 常量，表示每一个人的id不同，但一旦赋值又是不能变化的

    public PersonA(String n, String s, int a, int id) {
        name = n;
        sex = s;
        age = a;
        ID = id; // 构造函数中赋值
    }

    public String toString() {
        String s =
    "姓名：" + name + "," + "性别：" + sex + "," + "年龄：" + age;
        return s;
    }
}
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

示例代码

```java
public class FinalTest {
    public static void main(String args[]) {
        final double PAI = 3.1415926; // 局部常量

        // final既可以修饰简单数据类型，也可以修饰符和数据类型
        final PersonA p1 = new PersonA("Tom", "M", 23, 001);
        PersonA p2 = new PersonA("Mary", "F", 20, 002);
        System.out.println("final p1:" + p1.toString());

        // p1=p2 //对final对象重新赋值会产生编译错误
        // 以下对final对象中的成员变量，重新赋指是可以的
        p1.name = p2.name;
        p1.sex = p2.sex;
        p1.age = p2.age;
        System.out.println("final p1:" + p1.toString());
    }
}
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

小结：使用最终方法，最终类增强程序的鲁棒性

- 将方法或者类声明为final型可以有效防止他人覆写该函数，或者继承于该类。**但是或许更重要的是，这么做可以“关闭”动态绑定。**

- 或者说，这么做便是告诉编译器：动态绑定是不需要的。于是编译器可以产生效率较佳的程序代码。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

```java
package com.buaa.edu;
public class EduBackground {

    String primarySchool;
    String secondarySchool;
    String juniorHSchool;
    String seniorHSchool;
    String university;

    public EduBackground() {

    }
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

```java
package com.buaa.edu;
public class Student extends Person {
    private final int stuNumber;
    private int score;
    private static final int BAN_JI=20210001;
    public Student() {
        stuNumber=(int)Math.random()*500;
        score=(int)Math.random()*100;
    }
//子类不能重写父类被final修饰的方法
//  public final void finalMethod() {
//      int i = 2;
//      System.out.println("finalMethod: i = " + i);
//  }
    public final void priFinalMethod() {
        System.out.println("Student:priFinalMethod");
    }
}
```

```java
package com.buaa.edu;
public class Person {
    private String name;
    private int age;
    private String gender;
    private final EduBackground edu = new EduBackground();
    public Person() {
    }
    // final修饰局部变量、修饰成员方法、修饰方法的参数
    // 修饰局部变量时，局部变量的值不能改变
    public void finalLocal() {
        final int i;
        final EduBackground edu = new EduBackground();
        i = 1;
        System.out.println("finalLocal: i = " + i);
    }
    // 修饰方法的参数时(简单数据类型)，参数i不能被修改
    public void finalArgs(final int i) {
        // i = 3;
        System.out.println("finalArgs: i = " + i);
    }
    // 修饰方法的参数时(复合数据类型)，不能指向新的位置
    public void finalArgs(final EduBackground edu) {
        // edu = new EduBackground();
        System.out.println("finalArgs: edu");
    }

    // 修饰成员方法时，成员方法不能被子类重写
    public final void finalMethod() {
        int i = 2;
        System.out.println("finalMethod: i = " + i);
    }

    private final void priFinalMethod() {
        System.out.println("Person:priFinalMethod");
    }

    public static void main(String[] args) {
        Person per = new Person();
        Student stu = new Student();
        Person per1 = stu;

        per.priFinalMethod();
        stu.priFinalMethod();
        per1.priFinalMethod();
    }
}
```

北京航空航天大學
COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

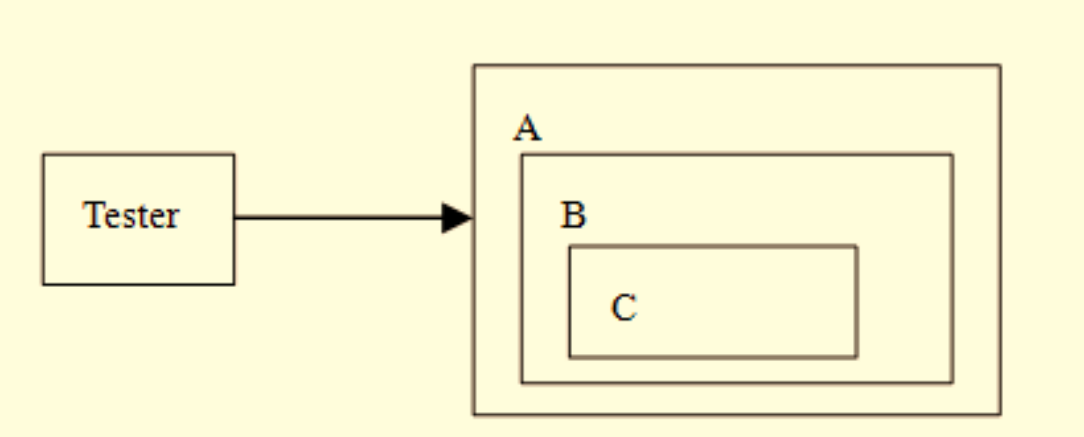
主要内容

- Object类
- 操作符“==”与对象的equals()方法
- 最终方法、最终类
- **内部类**
- **匿名类**

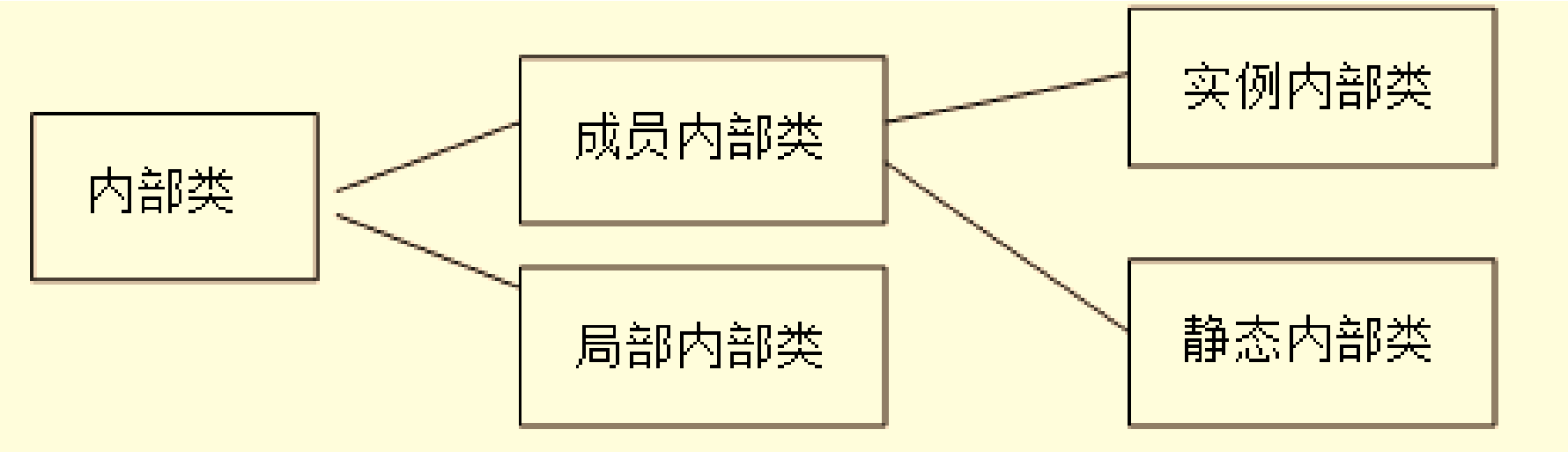
北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

主要内容

- 内部类的基本语法
  - 实例内部类
  - 静态内部类
  - 局部内部类
- 匿名类
- 内部类的用途
  - 内部类是一种封装类型



内部类的分类



示例代码

```java
class Outer {
    public class InnerTool { // 内部类
        public int add(int a, int b) {
            return a + b;
        }
    }
    private InnerTool tool = new InnerTool();

    public void add(int a, int b, int c) {
        tool.add(tool.add(a, b), c);
        System.out.println(tool.add(tool.add(a, b), c));
    }
}
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

示例代码

```
public class Tester {
    public static void main(String args[]) {
        Outer o = new Outer();
        o.add(1, 2, 3);
        Outer.InnerTool tool = new Outer().new InnerTool();
    }
}
```

在创建实例内部类的实例时，外部类的实例必须已经存在

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

创建实例内部类的实例

- 在创建实例内部类的实例时，外部类的实例必须已经存在
  - 例如：要创建InnerTool类的实例，必须先创建Outer外部类的实例
  - `Outer.InnerTool tool=new Outer().new InnerTool();`
  - 以上代码等价于：
  - `Outer outer=new Outer();`
  - `Outer.InnerTool tool =outer.new InnerTool() ;`

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

创建实例内部类的实例

- 以下代码会导致编译错误：
- `Outer.InnerTool tool=new Outer.InnerTool();`

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

实例内部类访问外部类的成员

- 在内部类中，可以直接访问外部类的所有成员，包括成员变量和成员方法。

- 实例内部类的实例自动持有外部类的实例的引用。

示例代码

内部类中，可以直接访问外部类的所有成员，包括成员变量和成员方法

```
public class A {
    private int a1;
    public int a2;
    static int a3;
    public A(int a1, int a2) {
        this.a1 = a1;
        this.a2 = a2;
    }
    protected int methodA() {
        return a1 * a2;
    }
    class B{  //内部类
        int b1=a1;  //直接访问private的a1
        int b2=a2;  //直接访问public的a2
        int b3=a3;  //直接访问static的a3
        int b4=new A(3,4).a1;  //访问一个新建的实例A的a1
        int b5=methodA();  //访问methodA()方法
    }
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

示例代码

```
    public static void main(String args[]){
        A.B b=new A(1,2).new B();
        System.out.println("b.b1="+b.b1);  //打印b.b1=1
        System.out.println("b.b2="+b.b2);  //打印b.b2=2
        System.out.println("b.b3="+b.b3);  //打印b.b3=0
        System.out.println("b.b4="+b.b4);  //打印b.b4=3
        System.out.println("b.b5="+b.b5);  //打印b.b5=2
    }
}
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

静态内部类

- 静态内部类的实例不会自动持有外部类的特定实例的引用，在创建内部类的实例时，不必创建外部类的实例。

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

示例代码

- 例如以下类A有一个静态内部类B，客户类Tester创建类B的实例时不必创建类A的实例：

```
class AA {
    public static class B {
        int v;
    }
}

class Tester {
    public void test() {
        AA.B b = new AA.B();
        b.v = 1;
    }
}
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

静态内部类

- 客户类可以通过完整的类名直接访问静态内部类的静态成员。

```
public class AAA {
    public static class B{
        int v1;
        static int v2;
        public static class C{
            static int v3;
            int v4;
        }
    }
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

示例代码

```
public class TesterAAA {
    public void test() {
        AAA.B b = new AAA.B();
        AAA.B.C c = new AAA.B.C();
        b.v1 = 1;
        b.v2 = 1;
        // AAA.B.v1=1; //编译错误
        AAA.B.v2 = 1; // 合法
        AAA.B.C.v3 = 1; // 合法
    }
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

局部内部类

- 局部内部类只能在当前方法中使用。
- 局部内部类和实例内部类一样，可以访问外部类的所有成员
- 此外，局部内部类还可以访问所在方法中的符合以下条件之一的参数和变量：
  - 最终变量或参数：用final修饰

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

局部内部类示例代码

```
public class AAAA {
    int a;
    public void method(final int p1, final int p2) {
        final int localV1 = 1;
        final int localV2 = 2;
        int localV3 = 0;
        localV3 = 1; // 修改局部变量
        class B {
            int b1 = a; // 合法，访问外部类的实例变量
            int b2 = p1; // 合法，访问final类型的参数
            int b3 = p2; // 合法，访问final类型的参数
            int b4 = localV1; // 合法,访问实际上的最终变量
            int b5 = localV2; // 合法，访问最终变量
            // int b6=localV3; //编译错误，localV3不是最终变量或者实际上的最终变量
        }
    }
}
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

匿名类

- 匿名类就是没有名字的类，是将类和类的方法定义在一个表达式范围里。
- 匿名类本身没有构造方法，但是会调用父类的构造方法。
- 匿名内部类将内部类的定义与生成实例的语句合在一起，并省去了类名以及关键字“class”,”extends”和“implements”等。

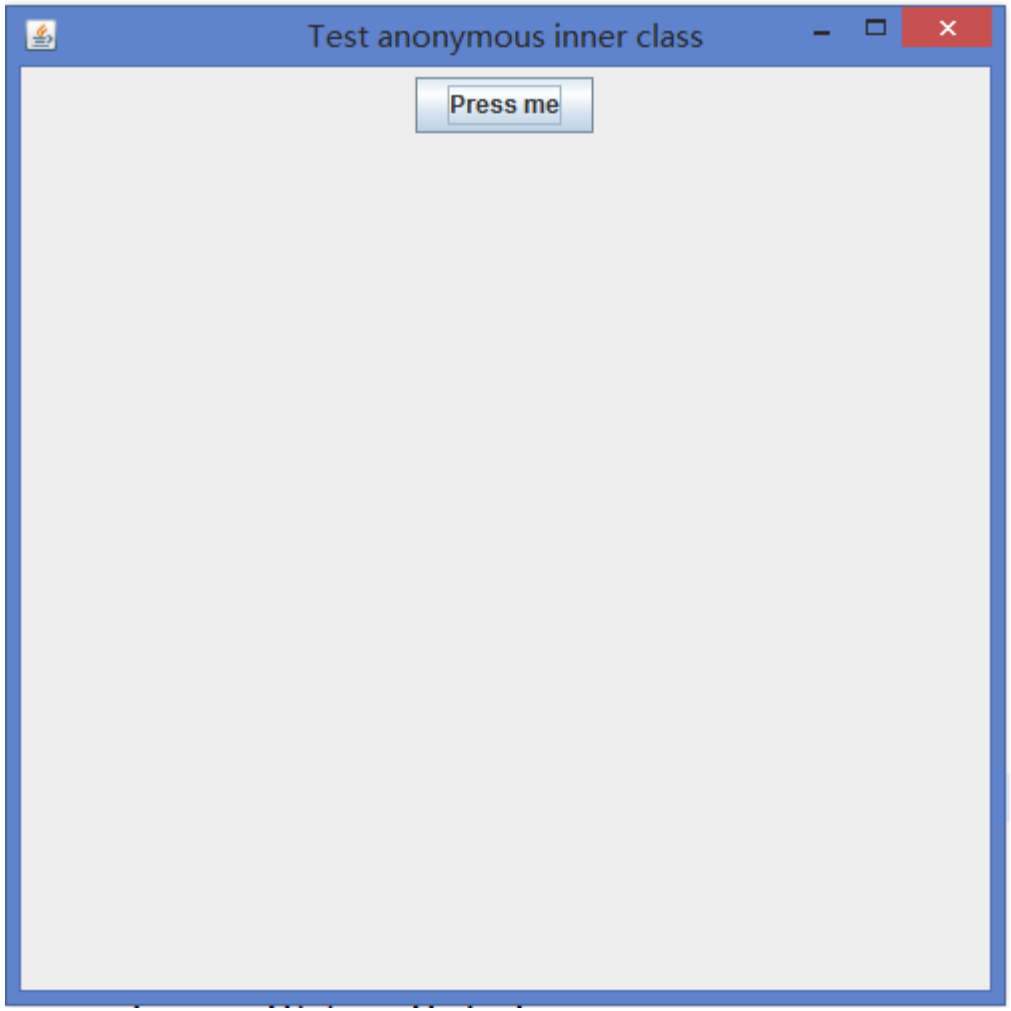
北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class J_Test1 extends JFrame{
  public J_Test1(){
    super("Test anonymous inner class");
    Container container=getContentPane();
    container.setLayout(new FlowLayout(FlowLayout.CENTER));
    JButton b=new JButton("Press me");
    container.add(b);
    b.addActionListener(
      new ActionListener(){
        public void actionPerformed(ActionEvent e){
          System.out.println("The button is pressed");
        }
    });     setSize(100,80);   setVisible(true);
  }
  public static void main(String[] args){
    J_Test1 application=new J_Test1();
    application.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
  }}
```

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class J_Test2 extends JFrame{
  public J_Test2(){
    super("Test anonymous inner class");
    Container container=getContentPane();
    container.setLayout(new FlowLayout(FlowLayout.CENTER));
    JButton b=new JButton("Press me");
    container.add(b);
    b.addActionListener(new J_ActionListener());
    setSize(100,80);    setVisible(true);
  }
  public static void main(String[] args){
    J_Test1 application=new J_Test1();
    application.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
  }}
class J_ActionListener implements ActionListener{
    public void actionPerformed(ActionEvent e){
      System.out.println("The button is pressed");
    }}
```

J_Test2.java

输出结果



示例代码

```java
public class AAAAA {
    AAAAA(int v) {
        System.out.println("another constructor");
    }
    AAAAA() {
        System.out.println("default constructor");
    }
    void method() {
        System.out.println("from AAAAA");
    };
    public static void main(String args[]) {
        new AAAAA().method(); // 打印from AAAAA
        AAAAA a = new AAAAA() { // 匿名类
            void method() {
                System.out.println("from anonymous");
            }
        };
        a.method(); // 打印from anonymous
    }
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

输出结果

```
default constructor
from AAAAA
default constructor
from anonymous
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

匿名类没有构造方法

```
    public static void main(String args[]) {
        int v = 1;
        AAAAA a = new AAAAA(v) { // 匿名类
            void method() {
                System.out.println("from anonymous");
            }
        };
        a.method(); // 打印from anonymous
    }

}
```

```
another constructor
from anonymous
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

内部类的用途（价值何在？）

- 封装类型
  - 如果一个类只能由系统中的某一个类访问，可以定义为该类的内部类。
- 直接访问外部类的成员
- 回调外部类的方法

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

内部类封装类型

- 顶层类只能处于public和默认访问级别
- 而成员内部类可以处于**public、protected、默认和private**四个访问级别。
- 此外，如果一个内部类仅仅为特定的方法提供服务，那么可以把这个内部类定义在方法之内。
- 可见，**内部类是一种封装类型的有效手段。**

内部类封装类型

```
public interface Tool {
    public int add(int a, int b);
}
```

```
public class Outer {
    private class InnerTool implements Tool {
        public int add(int a, int b) {
            return a + b;
        }
    }
    public Tool getTool() {
        return new InnerTool();
    }
    public static void main(String[] args){
        //InnerTool实例向上转型为Tool类型
        Tool tool=new Outer().getTool();
        Tool tool1=new Outer().new InnerTool();
    }
}
```

内部类封装类型（续）

- 在客户类中不能访问Outer.InnerTool类，但是可以通过Outer类的getTool()方法获得InnerTool的实例

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

内部类访问外部类的成员

- 内部类的一个特点是能够访问外部类的各种访问级别的成员。
- 假定有类A和类B，类B的reset()方法负责重新设置类A的实例变量count的值。
  - 一种实现方式是把类A和类B都定义为外部类：

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

示例代码

```
class A{
  private int count;
  public int add(){return ++count;}

  public int getCount(){return count;}

  public void setCount(int count){
        this.count=count;}
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

示例代码

```
class B{
  A a;   //类B与类A关联
  B(A a){this.a=a;}
  public void reset(){
    if(a.getCount()>0)
      a.setCount(1);
    else
     a.setCount(-1);
  }
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

内部类访问外部类的成员

- 假如需求中要求类A的count属性不允许被除类B以外的其他类读取或设置，那么以上实现方式就不能满足这一需求。
- 在这种情况下，把类B定义为内部类就可以解决这一问题，而且会使程序代码更加简洁

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

示例代码

```
class A{
  private int count;
  public int add(){return ++count;}

  class B{ //定义内部类B
    public void reset(){
      if(count>0)
        count=1;
      else
       count=-1;
    }
  }
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

回调

- 在以下Adjustable接口和Base类中都定义了adjust()方法，这两个方法的参数签名相同，但是有着不同的功能。

```
public interface Adjustable{
  /** 调节温度 */
  public void adjust(int temperature);
}
public class Base{
  private int speed;
  /** 调节速度 */
  public void adjust(int speed){
    this.speed=speed;
  }
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

回调

- 如果有一个Sub类同时具有调节温度和调节速度的功能，那么Sub类需要继承Base类，并且实现Adjustable接口，但是以下代码并不能满足这一需求：

adjust(int speed)
调节速度

```
public class Sub extends Base implements Adjustable{
  private int temperature;
  public void adjust(int temperature){
    this.temperature=temperature;
  }
}
```

adjust(int temerature)
调节温度

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

回调

```
public class Sub extends Base {
  private int temperature;
  private void adjustTemperature(int temperature){
    this.temperature=temperature;
  }
  private class Closure implements Adjustable{
    public void adjust(int temperature){
      adjustTemperature(temperature);
    }
  }
  public Adjustable getCallBackReference(){
    return new Closure();
  }
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

回调

- 以下代码演示客户类使用Sub类的调节温度的功能：

```
public class TestClass {
    public static void main(String[] args){
        //调节温度
        Sub sub=new Sub();
        Adjustable ad=sub.getCallBackReference();
        ad.adjust(15);

        //调节速度
        sub.adjust(350);
    }
}
```

接口回调

回调

- 回调实质上是指一个类尽管实际上实现了某种功能，但是没有直接提供相应的接口，客户类可以通过这个类的内部类的接口来获得这种功能。而这个内部类本身并没有提供真正的实现，仅仅调用外部类的实现。
- 可见，回调充分发挥了内部类具有访问外部类的实现细节的优势。

```
public class Sub extends Base {
  private int temperature;

  private void adjustTemperature(int temperature){
    this.temperature=temperature;
  }

  private class Closure implements Adjustable{
    public void adjust(int temperature){
      adjustTemperature(temperature);
    }
  }
  public Adjustable getCallBackReference(){
     return new Closure();
  }
}
```

内部类的文件

- 对于每个内部类，Java编译器会生成独立的.class文件。这些类文件的命名规则如下：
  - 成员内部类：外部类的名字$内部类的名字
  - 局部内部类：外部类的名字$数字和内部类的名字
  - 匿名类：外部类的名字$数字

```java
public class AAAAAA {
    static class B {
    } // 成员内部类，对应A$B.class
    class C { // 成员内部类，对应A$C.class
        class D {
        } // 成员内部类，对应A$C$D.class
    }
    public void method1() {
        class E {
        } // 局部内部类1，对应A$1E.class

        B b = new B() {
        }; // 匿名类1，对应A$1.class
        C c = new C() {
        }; // 匿名类2，对应A$2.class
    }
    public void method2() {
        class E {
        } // 局部内部类2，对应A$2E.class
    }
}
```

Java编译器编译以上程序，会生成以下类文件：

A.class

A$B.class

A$C.class

ACD.class

A$1E.class

A$1.class

A$2.class

A$2E.class

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

小结（思考题，下面的描述是否正确？）

比较方面	实例内部类	静态内部类	局部内部类
主要特征	内部类的实例引用特定的外部类的实例	内部类的实例不与外部类的任何实例关联	可见范围是所在的方法
可用的修饰符	访问控制修饰符，abstract,final	访问控制修饰符，static,abstract,final	abstract,final
可以访问外部类的哪些成员	可以直接访问外部类的所有成员	只能直接访问外部类的静态成员	可以直接访问外部类的所有成员，并且能访问所在方法的最终或实际上的最终变量和参数
拥有成员的类型	只能拥有实例成员	可以拥有静态成员和实例成员	只能拥有实例成员
外部类如何访问内部类的成员	必须通过内部类的实例来访问	对于静态成员，可以通过内部类的完整类名来访问	必须通过内部类的实例来访问

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

- 茶歇